

RobotStudio: Exercise 2

Using a gripper tool

Applied Robotics & Applied Robotics for Architects
FRTN85/FRTN80

Faculty of Engineering, Lund University

September 2025

Contents

1	Introduction	2
2	Simulated workstation setup	3
3	Creating paths for pick-and-place operations	7
3.1	Creating the first path	8
3.2	Setting up I/O connections	9
3.3	Using I/O signals on the path	12
4	Simulation	14
4.1	Seeing signals during the simulation	14
4.2	Modifying the simulator to visualize the pick-and-place	15
5	Copy-pasting paths	16

1 Introduction

In this exercise you will program a robot that picks objects from pre-specified locations and place them at other pre-specified locations. This operation is often referred to as “pick-and-place”. By doing so, you will learn the following general skills:

- basics of moving objects in a simulation environment
- working with digital outputs

You will be using the same robot, ABB IRB140, as last time.

The text of this exercise is less detailed and verbose than the first exercise. You will be given detailed instructions regarding **new** information, but concepts learned in the previous exercise will be referred to only on a high level.

Remember to work locally on the C:drive and save your work **FREQUENTLY!** The program is loaded through network drives and it can stop working unexpectedly. In between session save your work via *Pack & Go*, as explained in the first exercise.

2 Simulated workstation setup

Create new folders and a template with the *IRB140 6kg 0.81m* robot like last time. Wait for *Controller status* in the lower right corner of the screen to turn green. Using *Import Library*, import the pre-defined objects used for the exercise. They are located in:

This PC\shared\Courses\control\FRTF20_Applied_Robotics\RobotStudio_exercises_2019\Lab2 Gripper

The names of the objects to import are

- *brick*
- *gripper*
- *piece12*
- *rob-table*

Set the position of the robot on the table by putting its z coordinate to 670 mm. Put the gripper onto the robot by dragging it to the robot in the *Layout* tab on the left side of the screen. It should look as depicted in Figure 1.

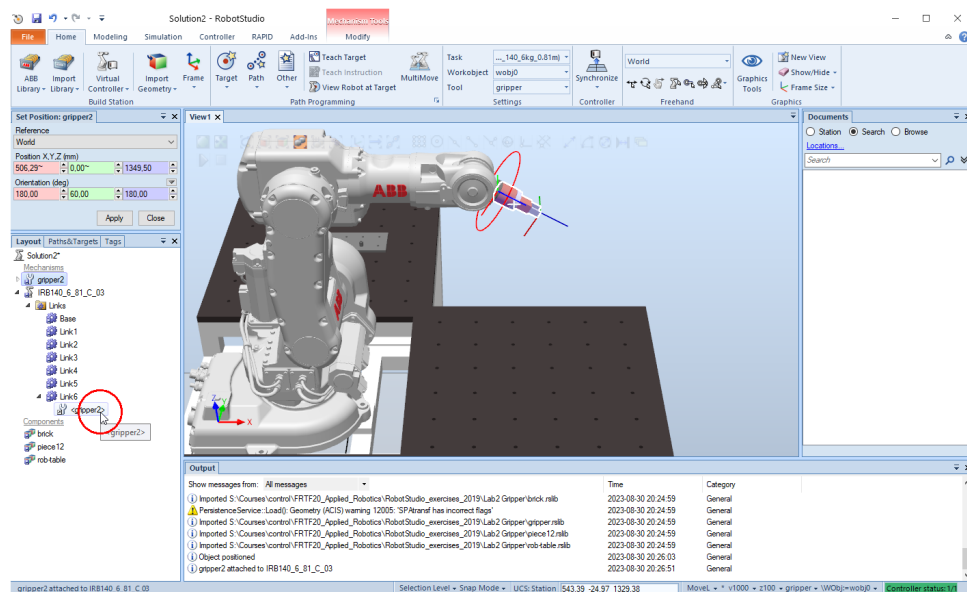


Figure 1: A correctly attached gripper

Next, define a workobject on the corner of the brick depicted in Figure 2. Make sure that the z axis is pointing down by applying a proper rotation. You will also need the following measurements:

- the distance between the holes is 50 mm in both x and y directions
- the distance to the first hole is 25 mm in both x and y directions
- the thickness of the plate is 8 mm

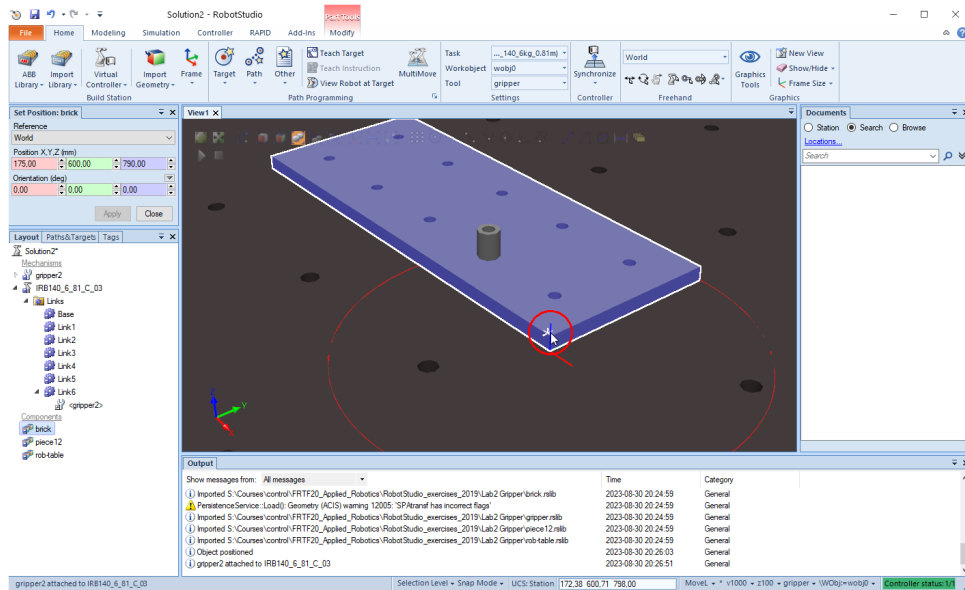


Figure 2: Define a workobject on the depicted corner

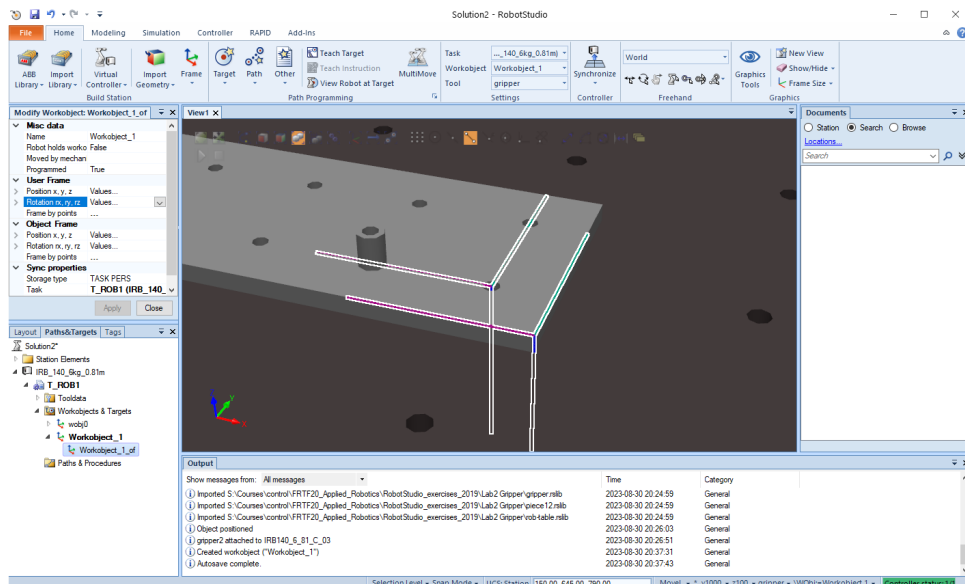
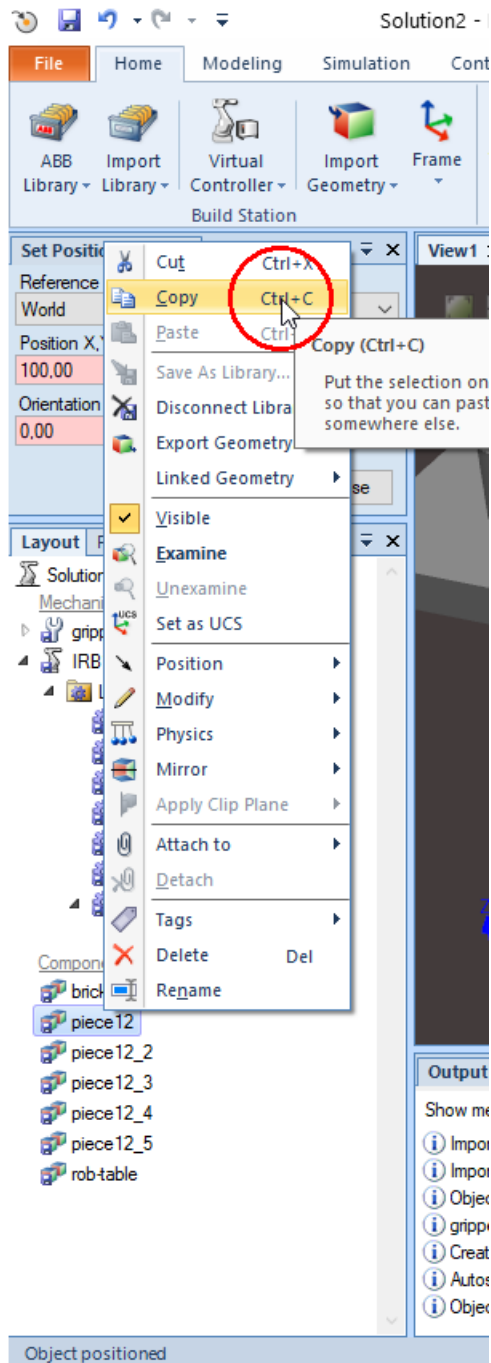


Figure 3: The desired workobject frames. The user frame is the one on the corner of the brick, while the object frame is in the hole closest to the user frame.

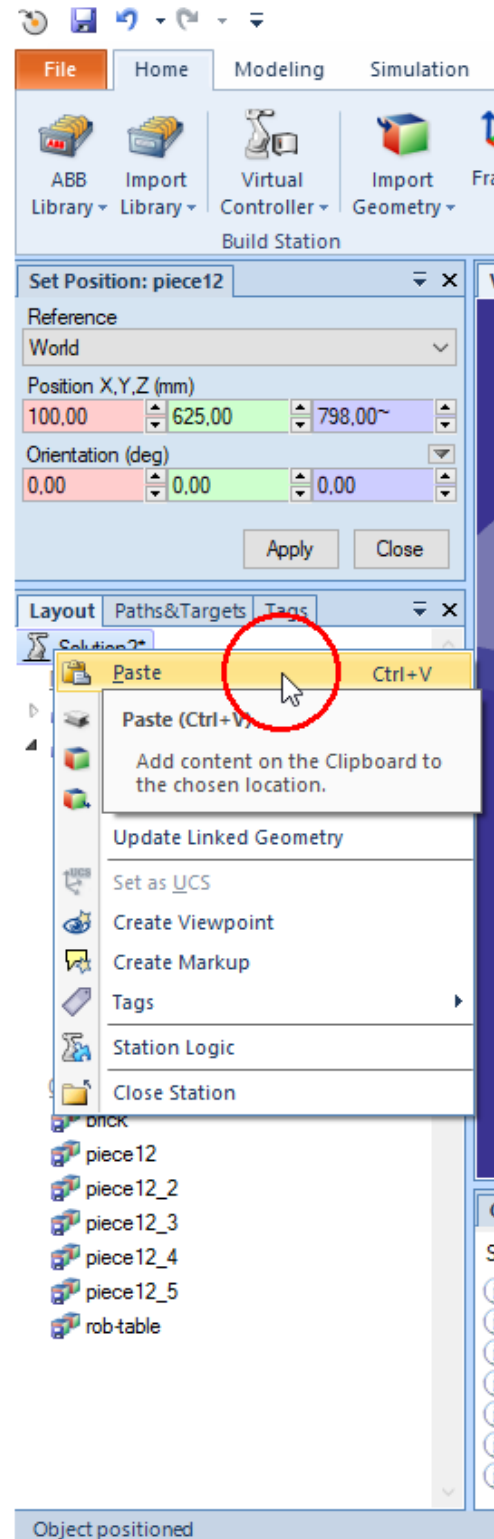
For the desired result refer to Figure 3. Remember that you can right-click on the workobject in the *Layout* tab and select *Modify Workobject* to change it.

Next, copy-paste the object “*piece12*”. For this task you will need 5 such objects in total. Right-click on the *piece12* and select *Copy*. Now right-click on the name of the station, which is the first item in the *Layout* tab, and select *Paste*. Refer to Figure 4 for a visual representation.

After copy-pasting, update the position of the objects by right clicking on one of them and selecting *Position* → *Set Position*. Once you have set the position of one of them (by putting in the values for the coordinates and clicking *Apply*), you can left-click on another one and its coordinates will appear in the *Set Position* tab (the one where you input coordinates). Put the appropriate coordinates for each of them so



(a) Selecting *Mechanism Joint Jog*



(b) Joint angle values for a proper home position

Figure 4: Copying and pasting *piece12*

that they fill the row of holes closer to the robot. Use the *Local* frame as *Reference* to help in the process. The result should look like what is depicted in Figure 5.

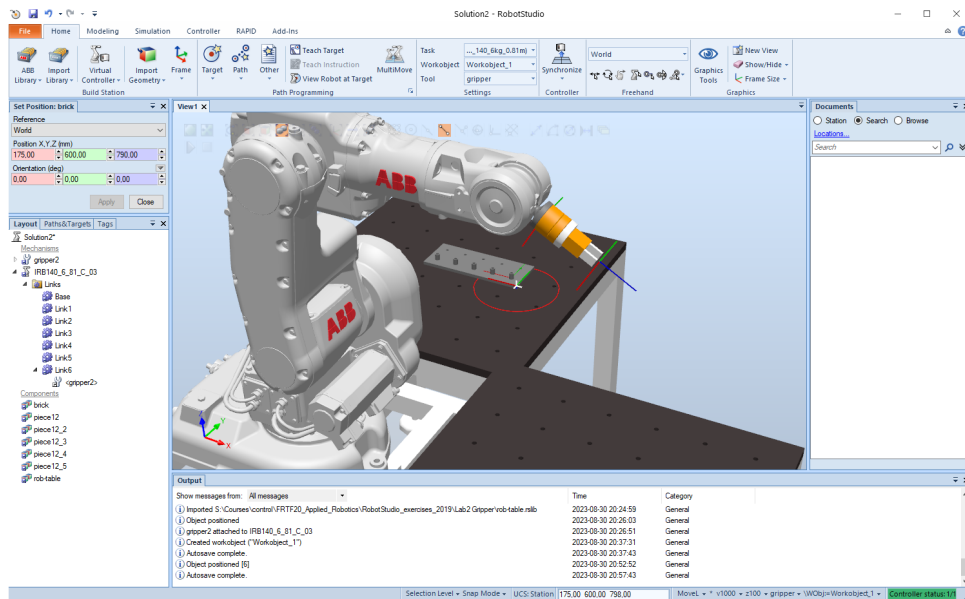


Figure 5: The desired positions of all 5 *piece12s*

This is a good time to save your current implementation.

3 Creating paths for pick-and-place operations

The task is to move each of the *piece12*s to the near hole in the empty row on the brick. While making the paths, ensure that the robot moves without hitting singularities or physical objects!

The general process for pick-and-place in RobotStudio is the following:

General pick-and-place process

1. move the tip of the robot (also know as the end-effector) to a target about 50 mm above a *piece12*
2. add a RAPID command to open gripper (to ensure it being open the first time and every time)
3. linearly move down to the *piece12*
4. add a RAPID command to close the gripper
5. move back up to the same target
6. go to a target above of the appropriate hole
7. move down
8. add a RAPID command to open the gripper
9. move back up
10. go to the next target above a *piece12* and repeat the process

In the proceeding text you will complete one path **for the first *piece12*** according to the general pick-and-place process outlined above. You can then copy-paste and translate that path to create the others, including their targets.

3.1 Creating the first path

To begin, make a target for the home position by using *Teach Target*. Be careful of the singularity the robot gets put into by default (jog the robot into a different position)! Next, create the target for the first *piece12* as described in the general pick-and-place process above. The result should look like Figure 6. Define the path and run *Auto Configuration*. Synchronize to RAPID. Save your implementation.

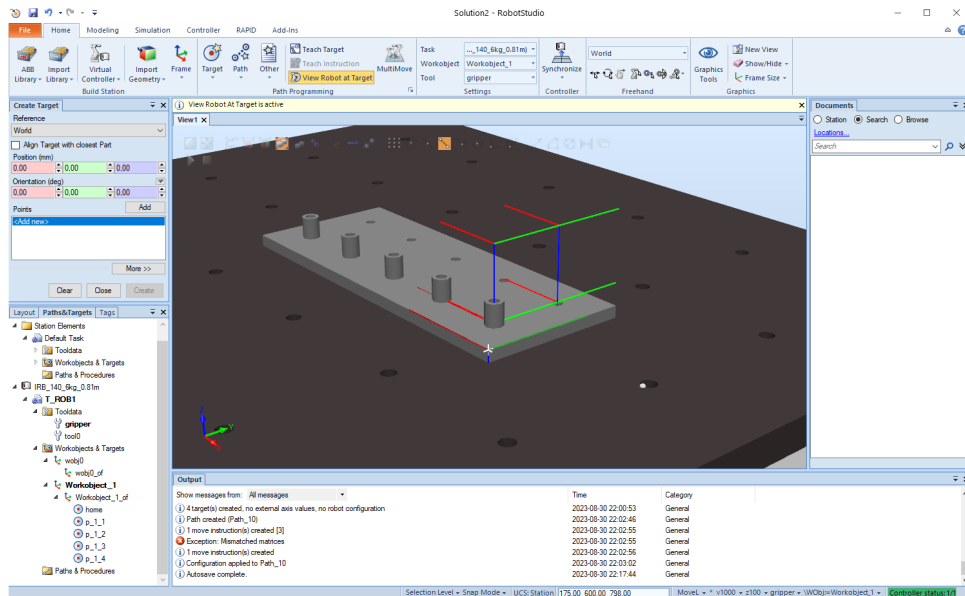


Figure 6: Target for the first pick-and-place path

Note: the motion from the home position to the first target should be **MoveJ**. The precise movements for pick-and-place should be *MoveL* (i.e., all other moves). For the non pick or place targets use velocity *v100* and zone *z10*. The pick and place targets should have a finer zone. What the zone means is explained graphically through Figure 7.

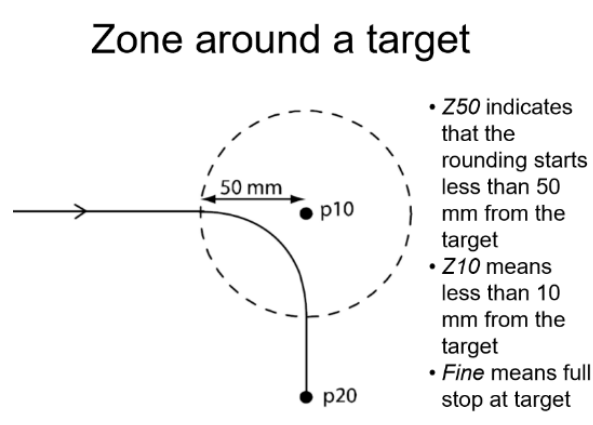


Figure 7: What the z-value, or zone means

Add your path to the *main* procedure if necessary. *Apply changes* and *Synchronize to Station*. Go to the *Simulation* tab in the menu on top of the screen and *Run* the simulation. As you will notice, **the piece12 will not move**. This is because you have

not yet programmed the opening and closing of the gripper by adding the appropriate digital I/O signals. In fact, you will need to add an additional simulation trick to see it happening, but we will get to that later.

3.2 Setting up I/O connections

You will now define and then use digital inputs and outputs (I/O connections) to open and close the gripper. In particular, you will use digital outputs. On our robot there are 16 digital outputs (DOs) and we refer to them as DO [ordinal number].

When DO 0 is set to 0, **and** DO 1 is set to 1, the gripper will **close**.
Alternatively, when DO 0 is set to 1, **and** DO 1 is set to 0, the gripper will **open**.

To use these signals, you must create **variables**, which will map to the mentioned **physical signals**.

It is **highly important** to use the same names for the signals as for the variables for your code to run properly on the real robot. In the code you will use variables named **DO10_0** and **DO10_1** to control the gripper.

To create these variables, go to the *RAPID* tab, expand the *Configuration* tree in the left-side menu, right-click on *I/O System* and then select *Configuration Editor*. This is depicted in Figure 8. Alternatively you can go to the *Controller* tab, click on *Configuration* and then selecting *I/O System*.

In the Configuration Editor, right-click on *Signal* on the left side of the editor and select *New Signal*. This is shown in Figure 9. Alternatively, go to the *Controller* tab, click on *Configuration* and select *Add Signals*.

After clicking on *New Signal*, you will get a pop-up window called *Instance Editor*. In it, specify the following:

- set *Name* to *DO10_0*
- set *Type of Signal* to *Digital Output*

Then do the same for DO10_1. **Make sure you have written these variable names correctly.**

For these changes to take effect, you will need to *Restart* by doing a *Warmstart*. You can see the location of the button in Figure 10.

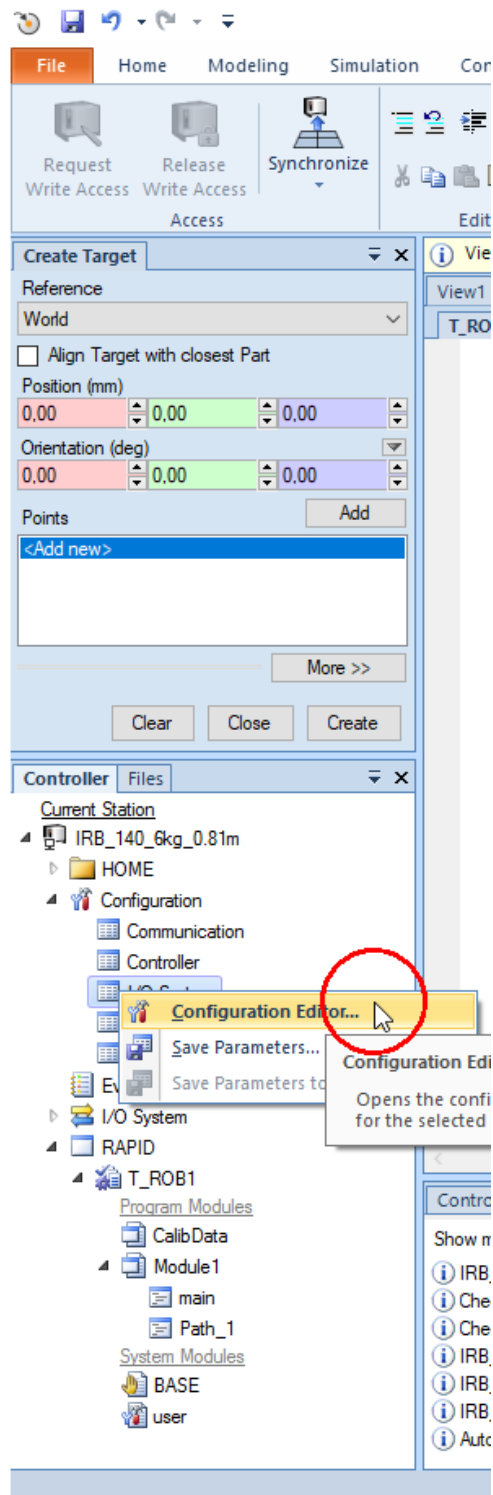


Figure 8: Locating the *Configuration Editor* for the *I/O System*

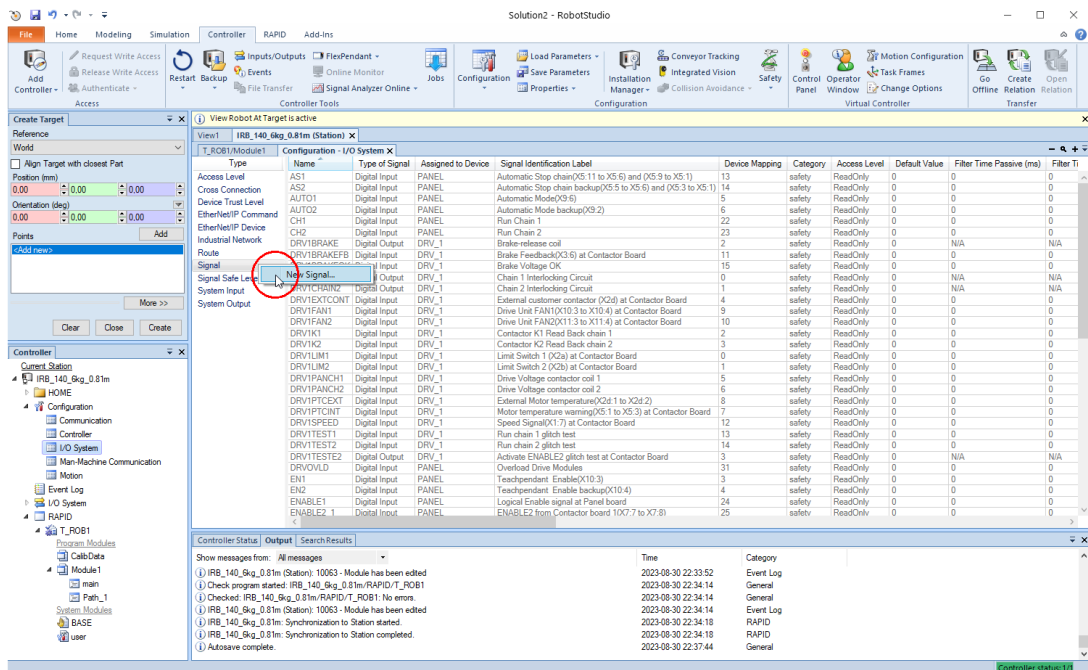


Figure 9: Selecting New Signal

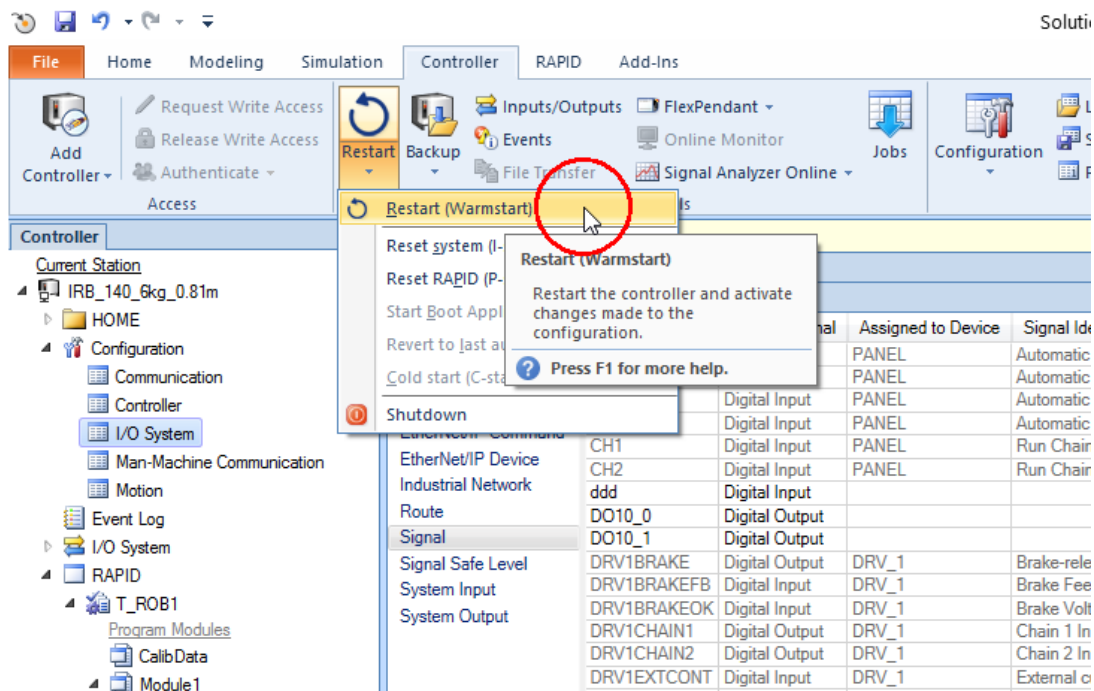
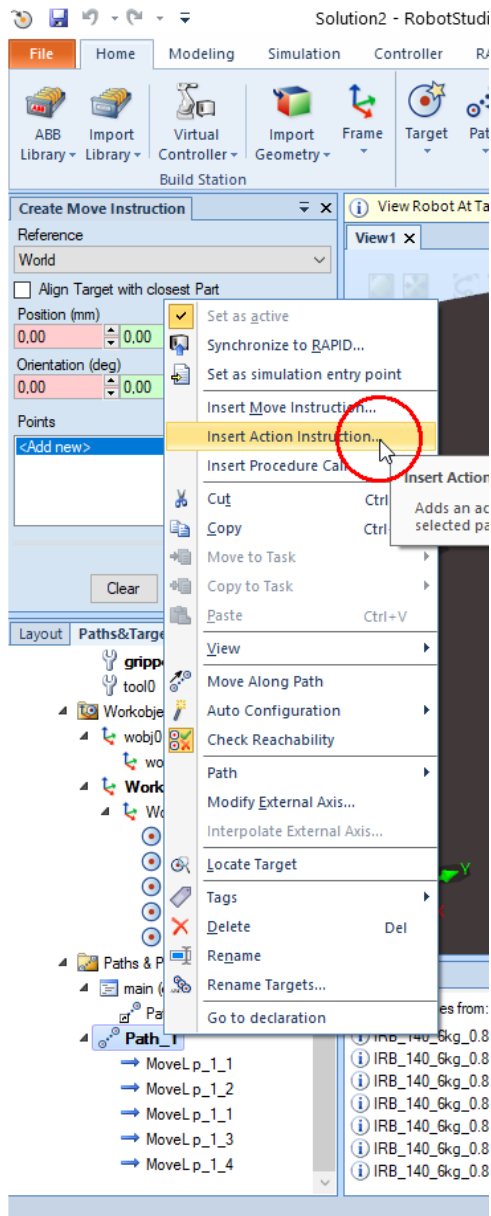


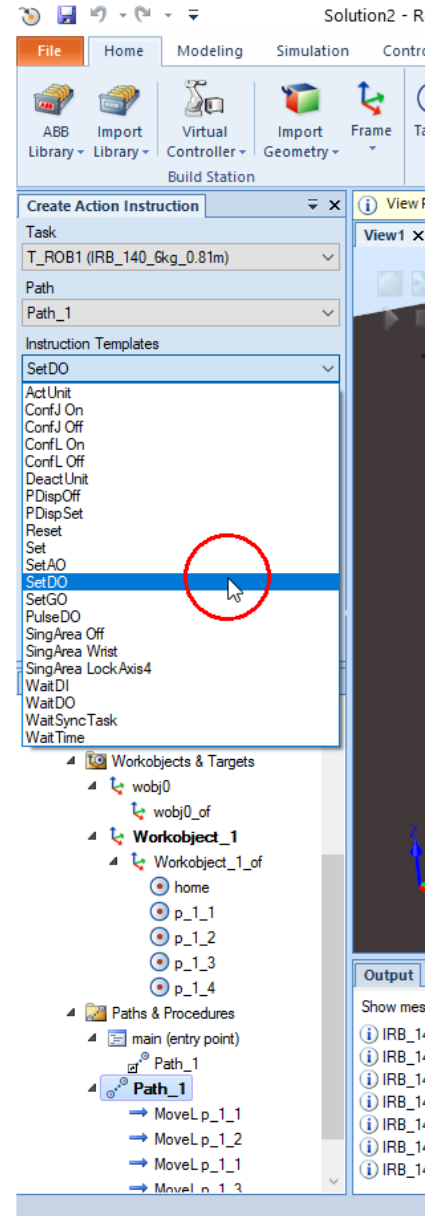
Figure 10: Location of the Warmstart button

3.3 Using I/O signals on the path

Now you can add these signals to your path. Right-click on the path and select *Insert Action Instruction*. To locate the button, consult Figure 11a. This will create a window above the *Paths & Targets* tab. In it, use the *SetDO* under *Instruction Templates* as depicted in Figure 11b.



(a) Location of the *Insert Action Instruction* button



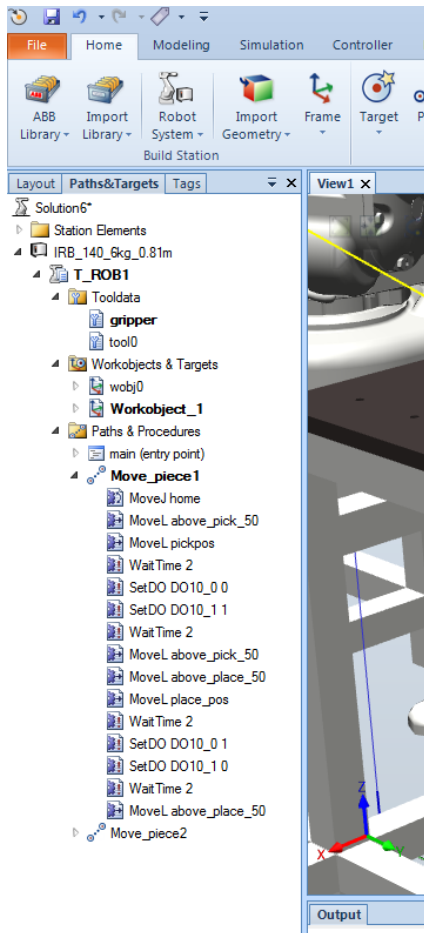
(b) Selecting *SetDO* under *Instruction Templates*

Figure 11: Inserting digital outputs via *Insert Action Instruction*

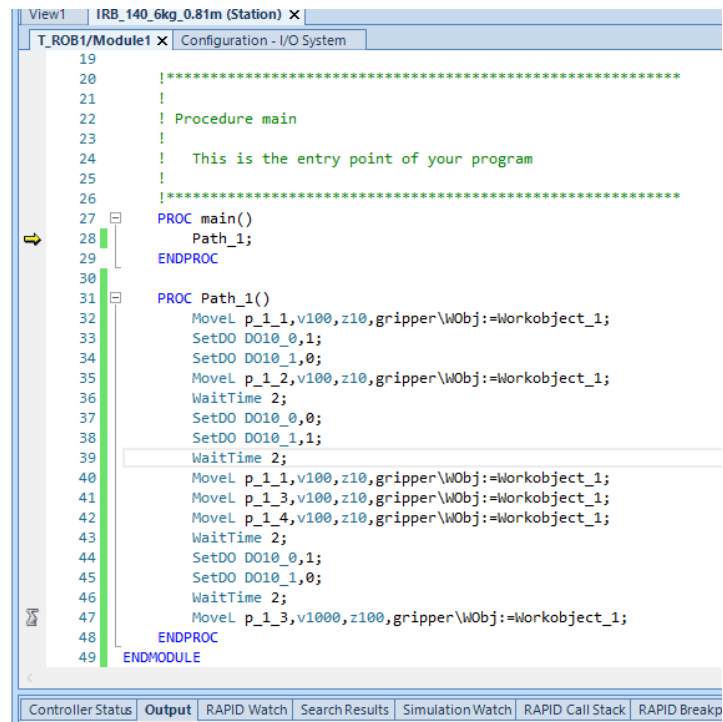
Add the appropriate *Actions* (set signals and values), and insert them into the path. Additionally, add a *WaitTime* of 2 seconds **before and after** each opening or closing action. This is necessary for the gripper to have time to move — the gripper uses a pneumatic system.

Remember that **both** DO10_0 and DO10_1 signals need to be set to appropriate values for the gripper to open or close. Do not forget to add the *WaitTime* actions.

The final path in the *Paths & Targets* tab should look something like Figure 12a, while an example in code can be seen in Figure 12b.



(a) An example of the final first path in the *Paths & Targets* tab



(b) An example of the final first path in RAPID code, here without the home target first

Figure 12: Final versions of the first path in the GUI and in the code

If you want your code to look better, create an *Open* and a *Close* procedure which encapsulates the 2 signals. This would make your code more readable.

4 Simulation

4.1 Seeing signals during the simulation

Once you have completed the first path, perform all the required *Apply* and *Synchronize* calls. Go to the *Simulation* tab and click on *I/O Simulator* in the top menu, as depicted in Figure 13. This will produce a tab of signals on the right side of the screen. There, under *Filter* select *Digital Outputs* as shown in Figure 14.

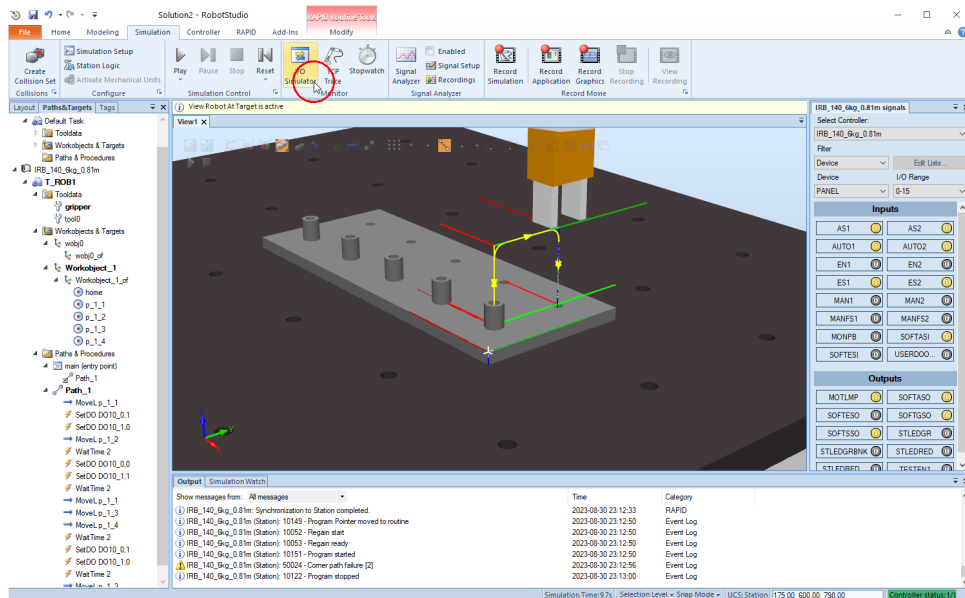


Figure 13: Location of the *I/O Simulator* button

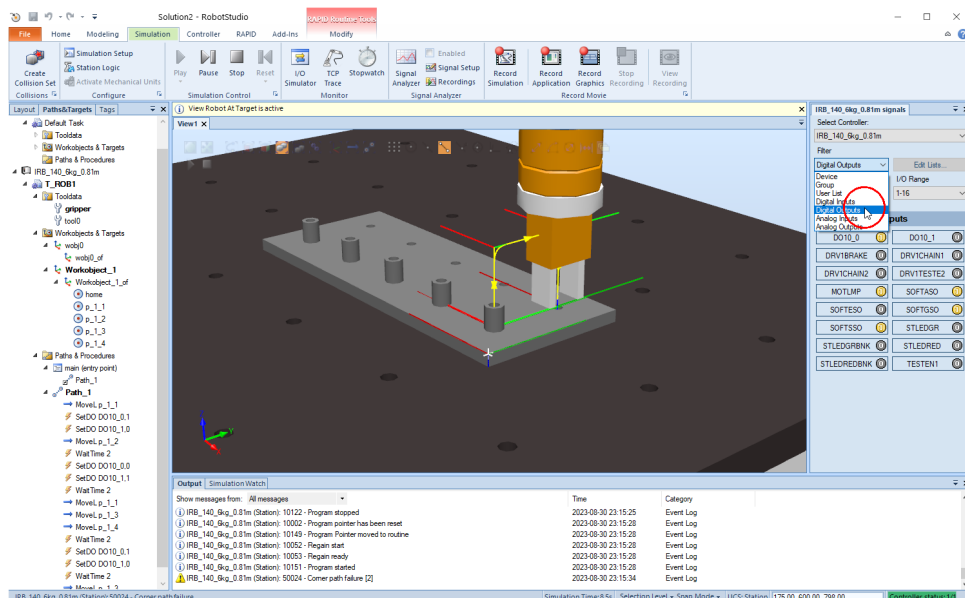


Figure 14: Selecting *Digital Outputs* under *Filter*

Run the simulation and observe the signals in the *I/O Simulator*.

4.2 Modifying the simulator to visualize the pick-and-place

As you would have noticed when running the simulation, the *piece12* still did not move! This is because RobotStudio is not a full physics simulator, but a robot programming tool. Consider the fact that even if there was a state-of-the-art physics simulator inside RobotStudio, it would still not be able to *exactly* emulate the real world — no simulator can. One always needs to rely on real-world experiments to generate real-world results.

Having this said, while we know that the signals will open and close the gripper on the real robot, it would be nice to have a visual confirmation that the robot does what we think it should before we perform the physical experiment. To achieve this, you will manually add the picking and placing in the RobotStudio simulation. **Note** that this will not change the code running on the real robot in any way.

To proceed, click on the *Event Manager* in the *Simulation* tab. It is a tiny arrow which you can find by consulting Figure 15.

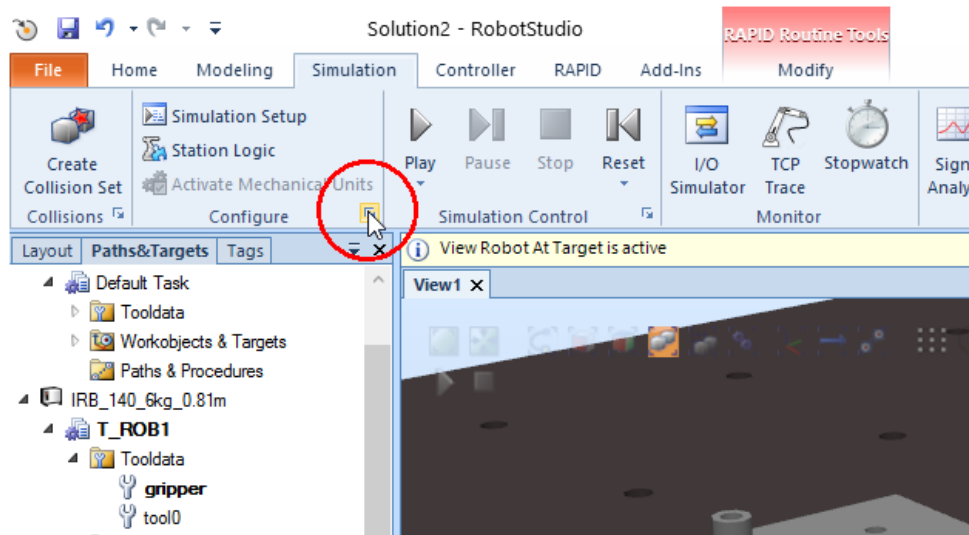


Figure 15: Location of the *Event Manager* button

This will lead you to the *Event Manager*. In it click on *Add*. You will get a pop-up window. In the wizard in the pop-up, perform the following steps:

1. Select *Activation* to be *On* and under *Event Trig Type* select *I/O Signal Changed*. Click on *Next*.
2. On the left-side select the signal *DO10_1*. Put *Signal Source* to *Active Controller* and the *Trigger Condition* to be *Signal is true ("1")*. Click on *Next*.
3. Under *Set Action Type* select *Attach Object*. Click on *Next*.
4. Under *Attach object* select *Find closest object*, and under *Attach to* select *gripper2*. Select the *Keep position* radio button. Click on *Finish*.

Go through the same process for *DO10_0*, but use *Detach object* under *Set Action Type*. Then, under *Detach object* select *Any object* and under *Detach from* select again select *gripper2*.

Once you have told the simulator to connect the signals to attachment and detachment of objects to the gripper, run the simulation. The *piece12* should now be picked up and placed.

Note: Set the position of the *piece12* back to the picking row so that everything works once you simulate again. Otherwise, the closest object will be the brick and it will be picked up!

5 Copy-pasting paths

Now that you have finished the path, and can validate it visually, you are ready to create paths for other *piece12*s. There is a number of ways to get this done. We will introduce a graphical one as it better serves as a part of the introduction to RobotStudio, but coding up the other paths with loops and more variables is probably the most convenient way in advanced use.

You will be copy-pasting and translating the path you have created for the first *piece12*, and doing so for all *piece12*s. This goes as follows:

Copy-pasting and translating a path

1. Go to the *Paths & Targets* tab, right-click on *Paths & Procedures* and select *Create Path*.
2. Open the original path you want to copy from and highlight all instructions (Mark the top instruction, hold down shift, then click the bottom instruction of the path). Right click on the marked instructions and select *Copy*.
3. Right click on the new empty path you created and select *Paste*.
4. In the pop-up confirm that you want to create new targets.
5. You will get a path identical to the copied one with new targets. Rename those new targets to something appropriate.
6. To translate the path, right-click on it and select *Path* → *Translate Path*.
7. You will get a tab in the same position, and similar in function to *Set position*. Define a vector by which you want to translate and hit *Apply* to move it on the next *piece12* and hole.
8. If you made a mistake, simply define and apply a new translation vector to fix your mistake.

Once you have created the paths, make sure to add home as the starting target of the first path, and last target of the last path. Your final view should look similar to Figure 16.

Make an *Auto Configuration* on all paths. Synchronize to RAPID. Modify the main function to include the new paths. **Check everything — speeds, zones and targets once more in the RAPID code.**

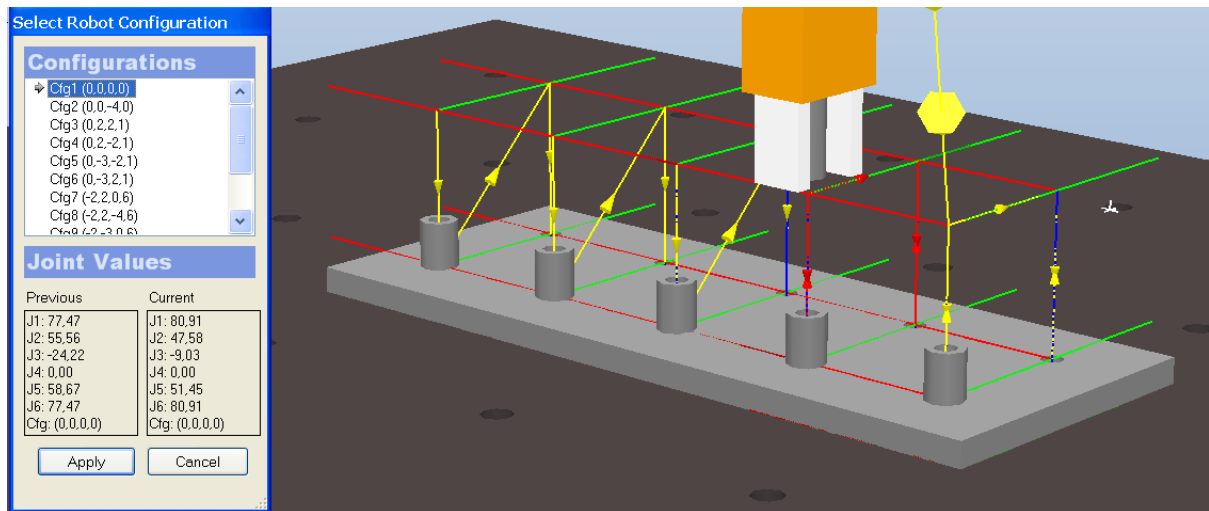


Figure 16: An example of the final version of all paths

Once you are satisfied with the simulation, save your program via the *RAPID* tab (top of screen) → *Controller* (left side of screen) → *RAPID* → *T_ROB1* → right-click and *Save As*. **Show the simulation to the teaching assistant.** You need to show the simulation **and** demonstrate on the real robot to pass the exercise.